Task-17:- Technical Documentation(Multi-VPC AWS network environment)

1. Introduction

1.1 Purpose

The purpose of this document is to describe the implementation of a **Multi-VPC AWS network architecture deployed across multiple AWS regions using VPC Peering**.

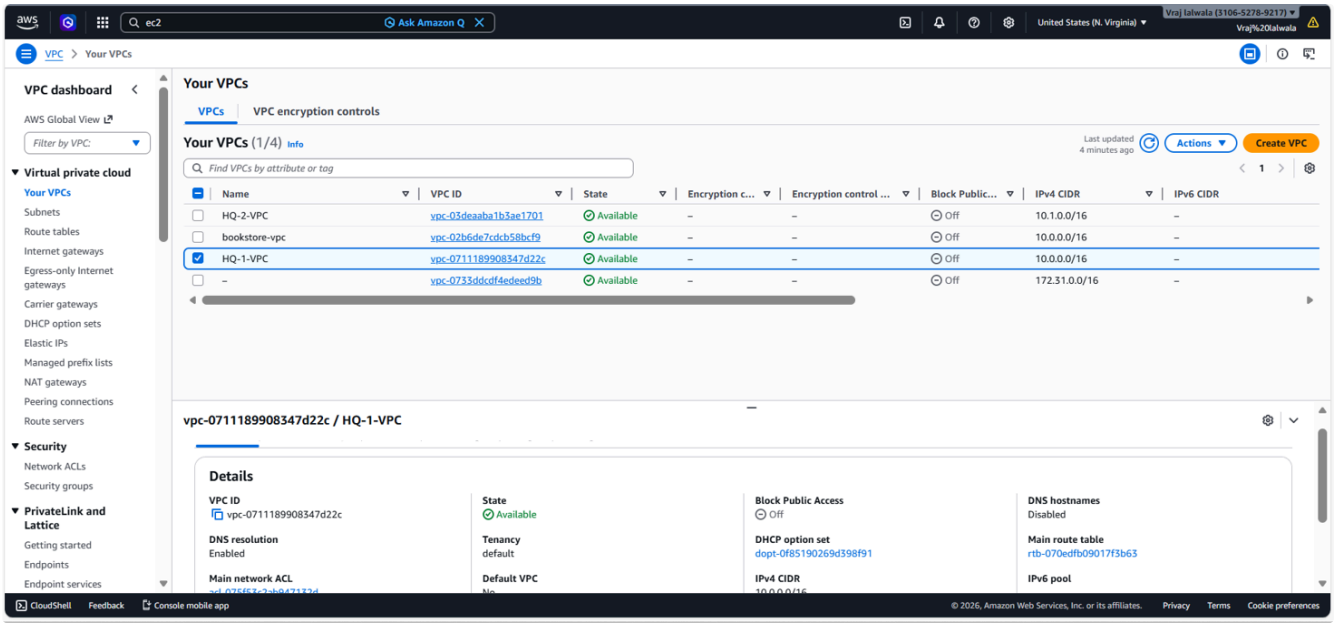
This document provides a detailed explanation of the infrastructure setup including:

- VPC creation
- Subnet planning
- Route table configuration
- VPC peering
- EC2 deployment
- Connectivity testing

The intended audience includes:

- CTO
- DevOps Engineers
- Solution Architects
- Developers
- Cloud Engineers
- Infrastructure Engineers

This document serves as a **step-by-step implementation guide** for creating scalable cloud network infrastructure.



1.2 Scope

This document describes the implementation of a **multi-region AWS network architecture** using the following components:

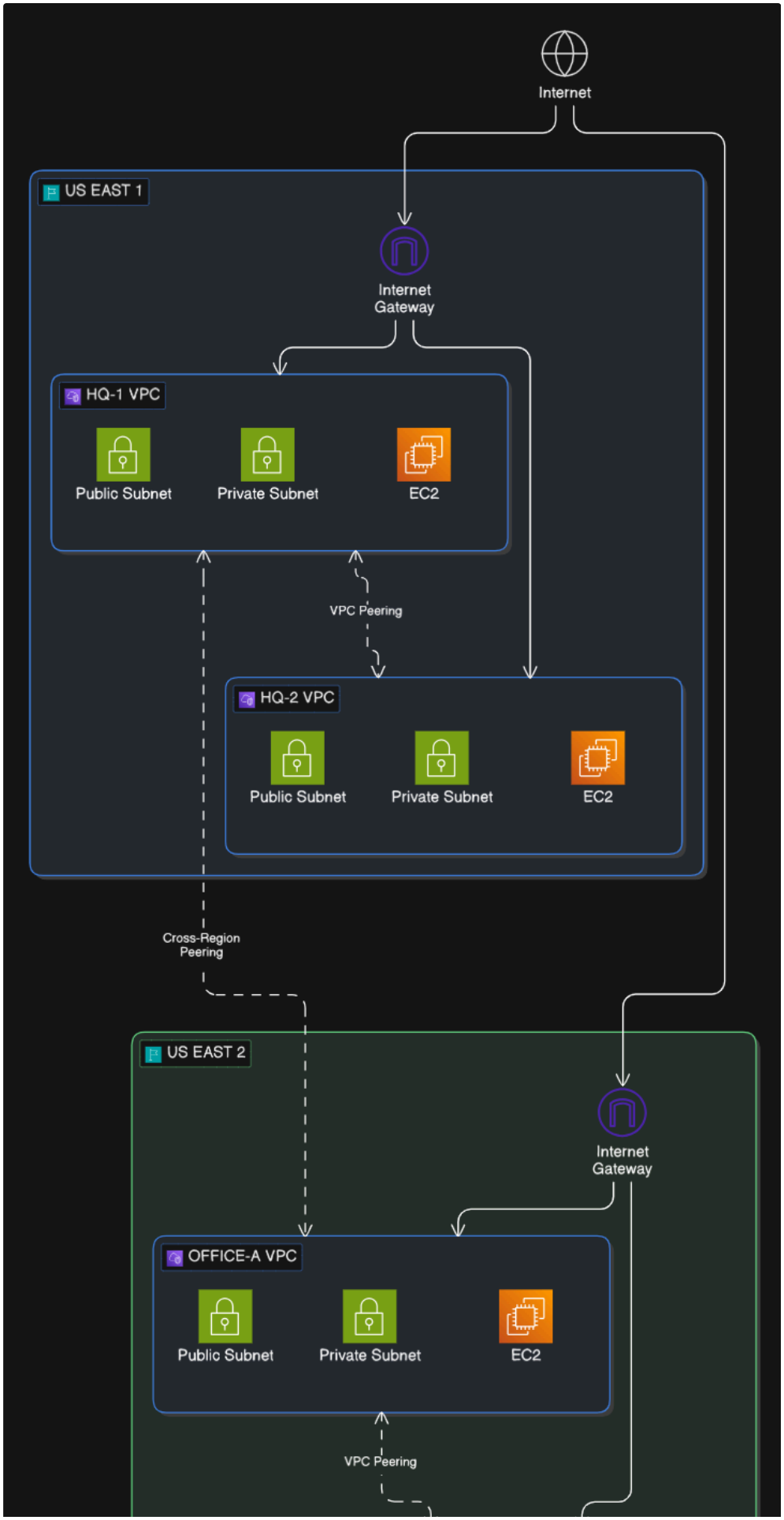
- Virtual Private Clouds (VPC)
- Public and Private Subnets
- Internet Gateways
- Route Tables
- VPC Peering Connections
- EC2 Instances

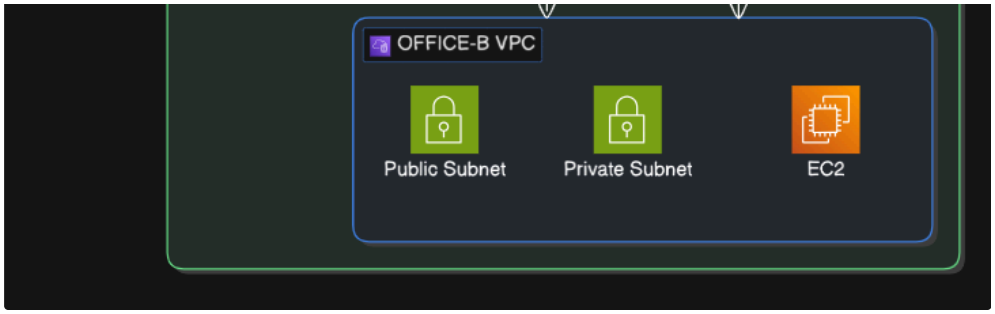
The scope includes:

- Network design and CIDR planning
- Infrastructure deployment
- Connectivity verification

The scope does **not include**:

- Production application deployment
- Database clustering
- AWS Transit Gateway configuration


Internet
US EAST 1
Internet Gateway
HQ-1 VPC
Public Subnet
Private Subnet
EC2
VPC Peering
HQ-2 VPC
Public Subnet
Private Subnet
EC2
Cross-Region Peering
US EAST 2
Internet Gateway
OFFICE-A VPC
Public Subnet
Private Subnet
EC2
VPC Peering



---

2. Overview of DevOps Project

2.1 Definition

DevOps integrates development and operations to enable faster and more reliable infrastructure deployment.

In this project, DevOps practices are used to design and deploy a **multi-VPC cloud network infrastructure using AWS services**.

Benefits include:

- Improved scalability
- Secure network segmentation
- Efficient infrastructure management
- Faster infrastructure deployment

2.2 Key Principles

This project follows the **AWS Well-Architected Framework**.

Operational Excellence

Infrastructure deployment is documented and repeatable.

Security

Private subnets are used for internal resources.

Reliability

Infrastructure is distributed across multiple VPCs and regions.

Performance Efficiency

Subnet sizes are designed according to host requirements.

Cost Optimization

Low-cost instances (t3.micro) were used.

Sustainability

Efficient resource allocation reduces unnecessary cloud usage.

2.3 Goals and Objectives

The objectives of this project include:

- Designing a **multi-VPC architecture**
- Implementing **advanced subnetting**
- Establishing **cross-region connectivity**
- Deploying **EC2 instances for testing**
- Verifying network communication between VPCs

2.4 Constraints and Assumptions

Constraints:

- Time limit: **3 hours**
- AWS services only
- CIDR blocks must not overlap

Assumptions:

- AWS account access is available
- Internet access is available for management
- Default AWS service limits are sufficient

---

3. Architecture and Infrastructure

3.1 Infrastructure Overview

The infrastructure consists of the following VPC environments.

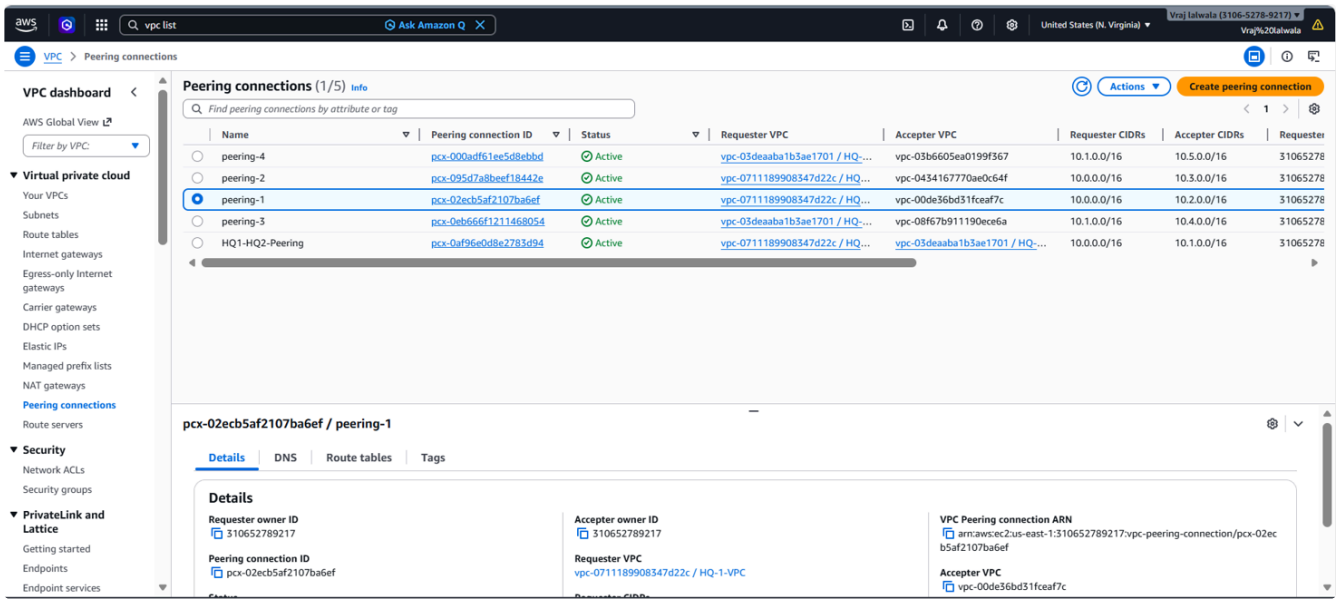
Region	VPCs
us-east-1	HQ-1 VPC, HQ-2 VPC
us-east-2	Office-A VPC, Office-B VPC, Office-C VPC, Office-D VPC

Each VPC contains:

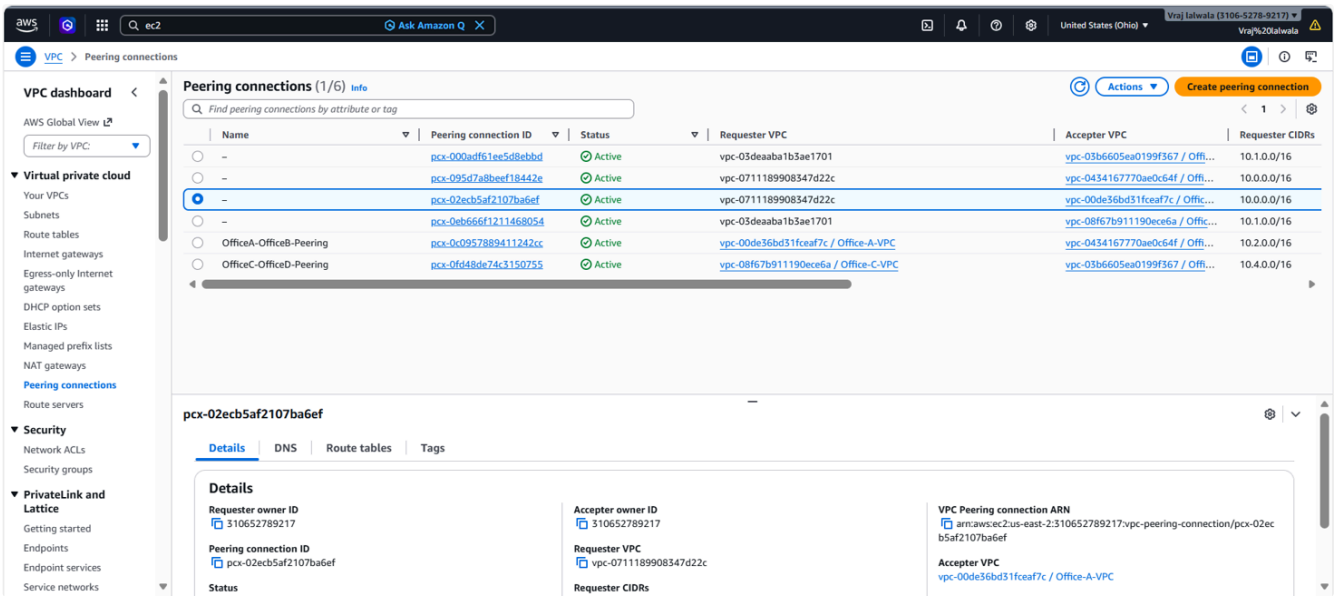
- Public subnet
- Private subnet
- Route tables
- Internet gateway

Connectivity between VPCs is implemented using **VPC Peering**.

HQ Environment VPC peering:-



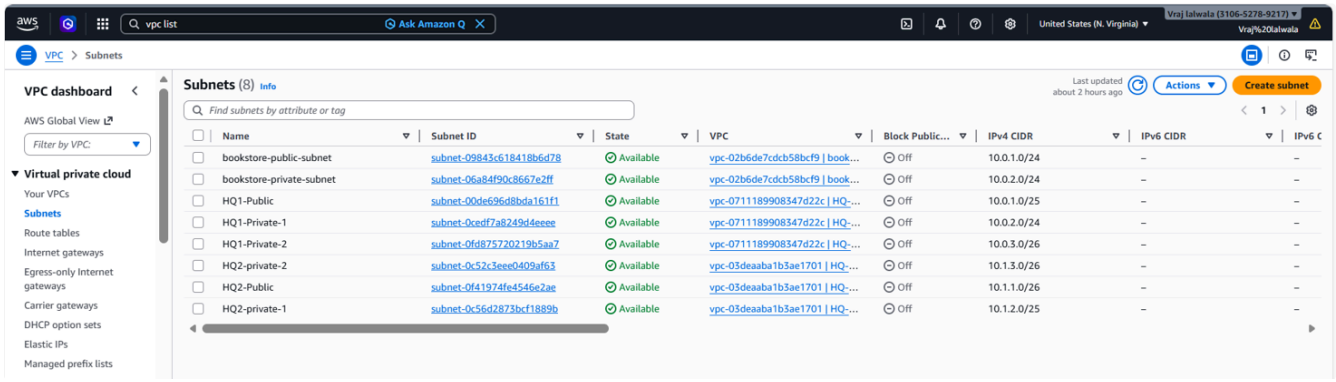
Branch Office VPC peering:-



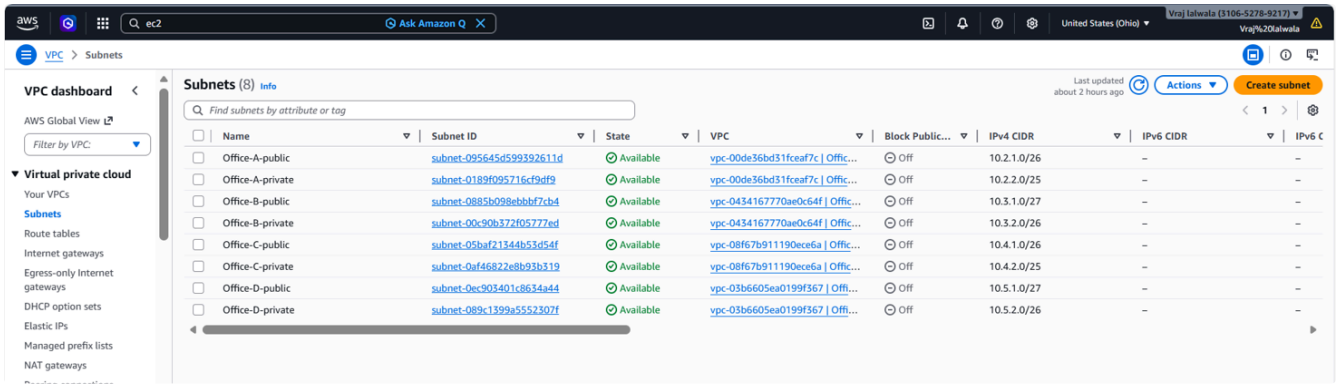
3.2 Environments

Environment	Purpose
HQ Networks	Corporate infrastructure
Branch Offices	Regional networks
Public Subnets	Internet-facing resources
Private Subnets	Internal application servers

HQ Subnets:-



Branch Office Subnets:-



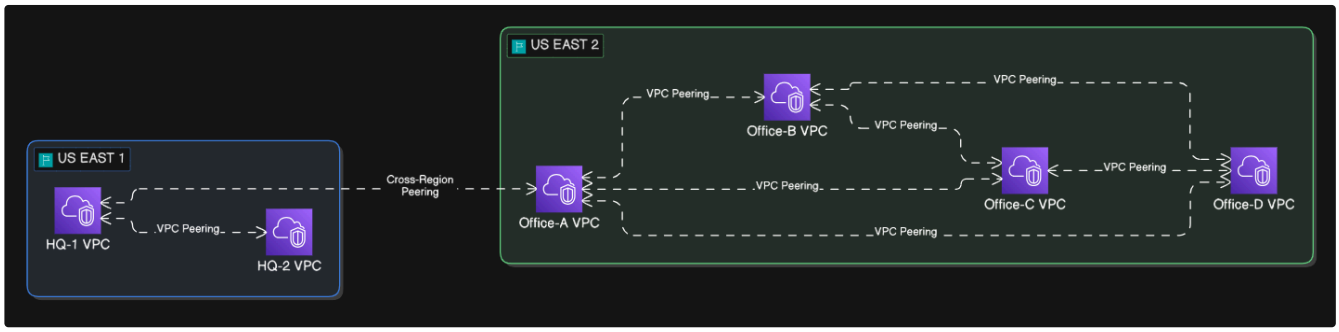
3.3 Tools and Technologies

Tool	Purpose
AWS VPC	Network infrastructure
AWS EC2	Compute resources
VPC Peering	Network connectivity
Route Tables	Traffic routing
Security Groups	Network security
Internet Gateway	Internet connectivity

3.4 Application Architecture & User Request Flow

User request flow:

1. User connects to EC2 instance in public subnet.
2. Traffic flows through Internet Gateway.
3. Internal communication occurs via VPC Peering.
4. Private resources communicate via internal IP addresses.



4. Development Practices

4.1 Version Control

Infrastructure documentation and configuration management can be maintained using Git repositories.

Example tools:

- GitHub
- GitLab
- Bitbucket

4.2 Continuous Integration

Typical DevOps CI tools include:

- GitHub Actions
- Jenkins
- AWS CodePipeline

These tools automate build and deployment pipelines.

4.3 Automated Testing

Network connectivity testing was performed using:

```
telnet <private-ip> 22
```

Example:

```
telnet 10.2.1.28 22
```

```
[root@ip-10-0-1-65 ec2-user]# telnet 10.0.1.65 22
Trying 10.0.1.65...
Connected to 10.0.1.65.
Escape character is '^]'.
SSH-2.0-OpenSSH_8.7
^C
Connection closed by foreign host.
[root@ip-10-0-1-65 ec2-user]# ping 10.0.1.65
PING 10.0.1.65 (10.0.1.65) 56(84) bytes of data.
64 bytes from 10.0.1.65: icmp_seq=1 ttl=127 time=0.026 ms
64 bytes from 10.0.1.65: icmp_seq=2 ttl=127 time=0.031 ms
64 bytes from 10.0.1.65: icmp_seq=3 ttl=127 time=0.029 ms
64 bytes from 10.0.1.65: icmp_seq=4 ttl=127 time=0.032 ms
^C
--- 10.0.1.65 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3109ms
rtt min/avg/max/mdev = 0.026/0.029/0.032/0.002 ms
[root@ip-10-0-1-65 ec2-user]# telnet 10.2.1.28 22
Trying 10.2.1.28...
telnet: connect to address 10.2.1.28: Connection timed out
[root@ip-10-0-1-65 ec2-user]# telnet 10.2.1.28 22
Trying 10.2.1.28...
Connected to 10.2.1.28.
Escape character is '^]'.
SSH-2.0-OpenSSH_8.7
```

4.4 Code Quality and Analysis

Best practices implemented:

- Non-overlapping CIDR blocks
- Public/private subnet separation
- Secure network communication
- Minimal open ports

---

5. Deployment and Release Management

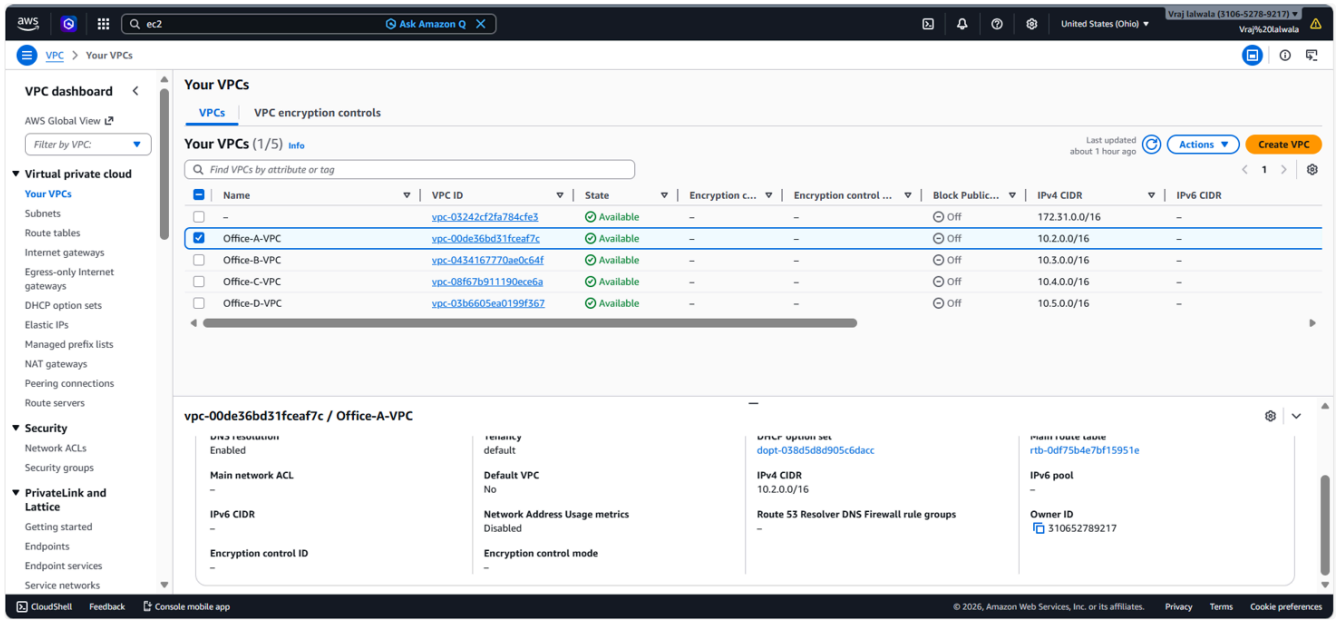
5.1 Deployment Strategy

Infrastructure deployment followed these steps:

1. Create VPC
2. Create subnets

3. Configure Internet Gateway
4. Configure route tables
5. Establish VPC peering
6. Deploy EC2 instances
7. Test connectivity

The **same process was repeated for all VPC environments**.



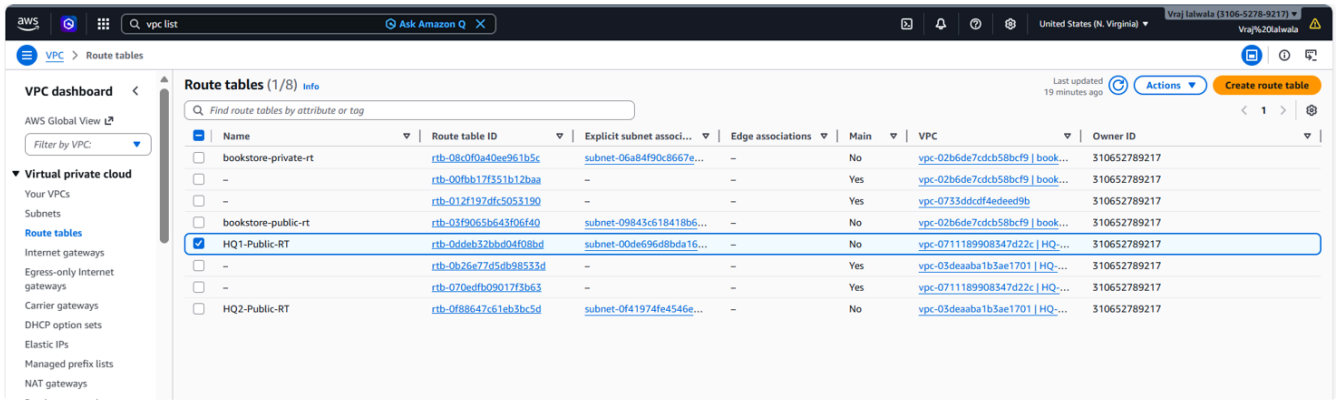
5.2 Configuration Management

Infrastructure configuration included:

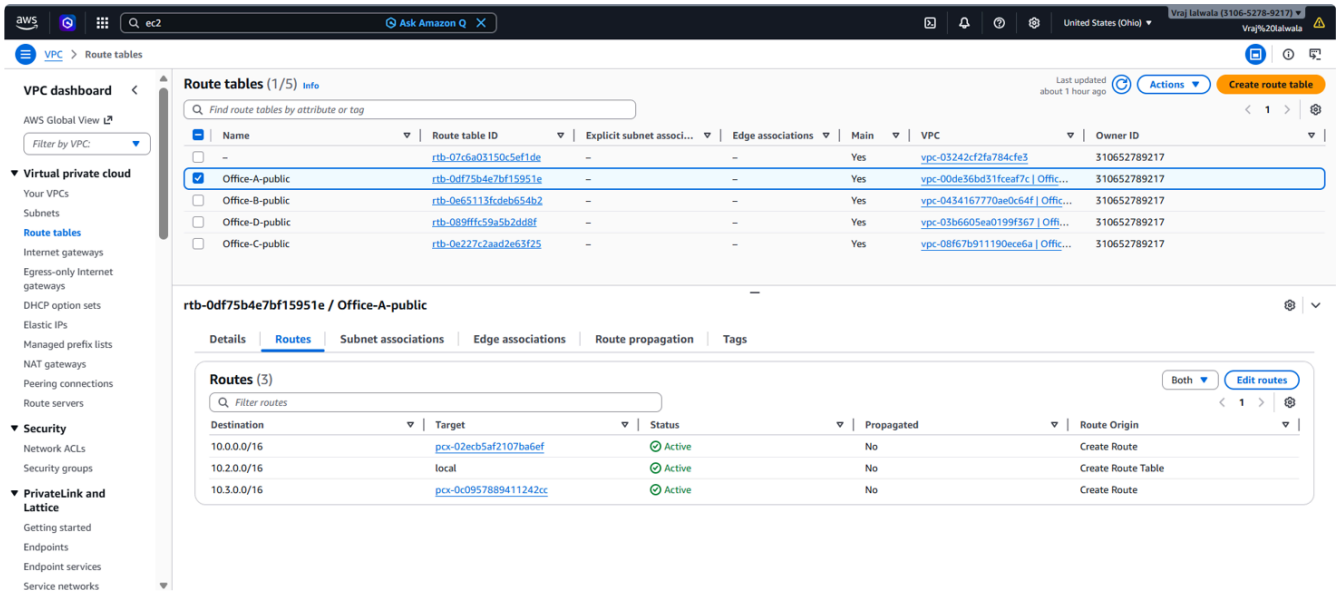
- CIDR block allocation
- Route table configuration
- Internet gateway attachment
- Security group configuration

These configurations were repeated for **each VPC**.

HQ Route tables:-



Branch Office Route tables:-



5.3 Continuous Delivery / Deployment

Infrastructure deployment was performed manually using AWS console.

Future automation could include:

- Terraform
- AWS CloudFormation

---

6. Monitoring and Incident Management

6.1 Monitoring Tools and Metrics

Monitoring tools available:

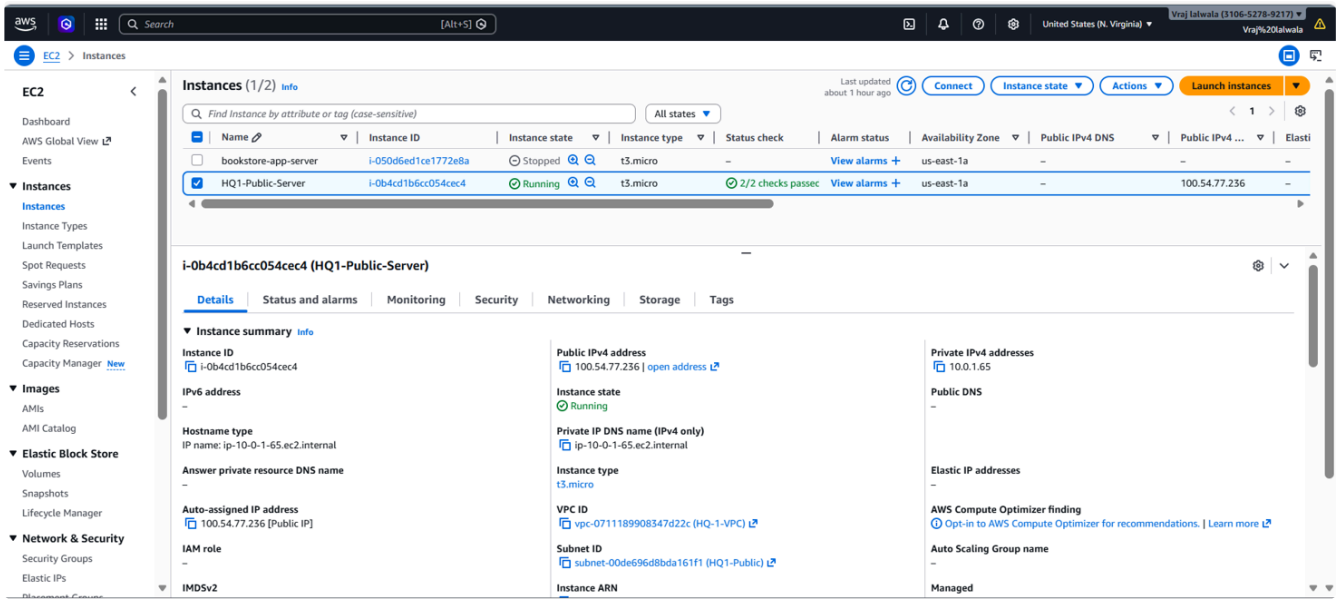
- AWS CloudWatch
- EC2 Status Checks

- VPC Flow Logs

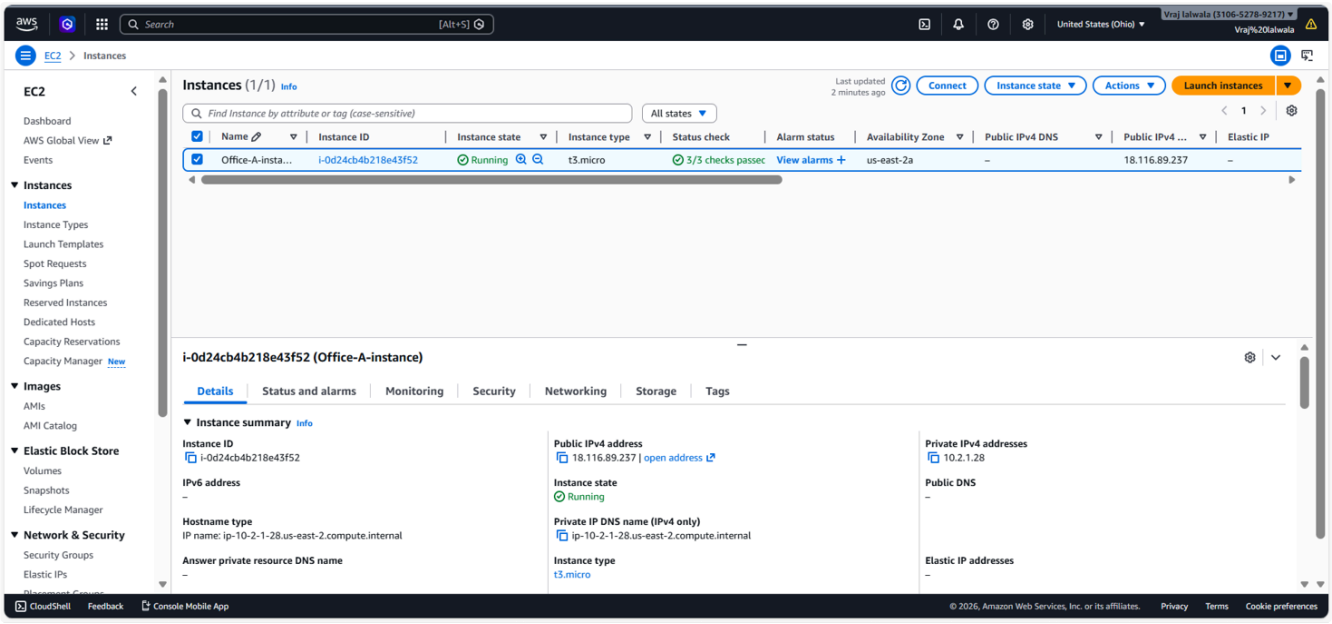
Key metrics monitored:

- Instance health
- Network traffic
- Connectivity status

HQ Environment Instance:-



Branch Office Instance:-



6.2 Alerting and Notification

CloudWatch alarms can be configured to notify administrators if infrastructure issues occur.

Examples:

- Instance failure alerts
- Network connectivity alerts

6.3 Incident Management

Incident resolution process:

1. Identify issue
2. Verify route tables
3. Verify security groups
4. Check VPC peering status
5. Resolve configuration issues

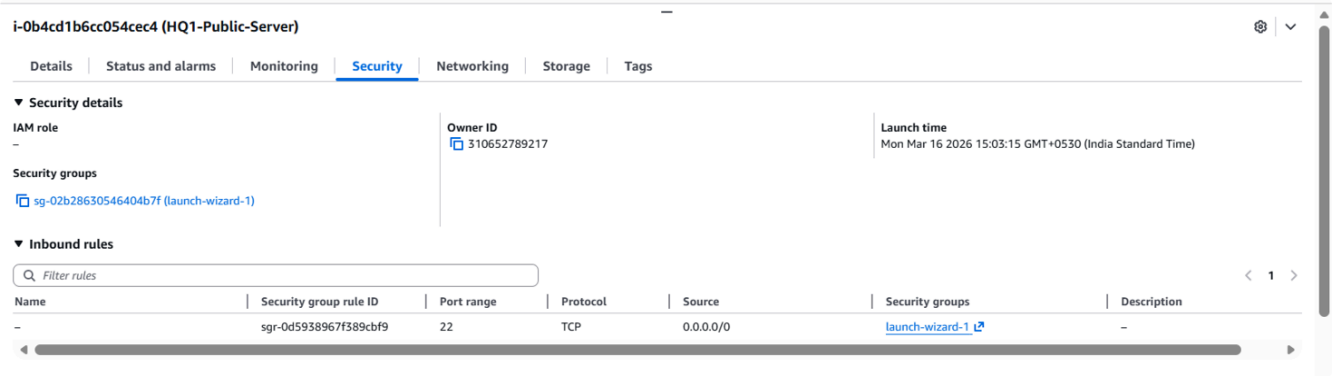
---

7. Security & Compliance

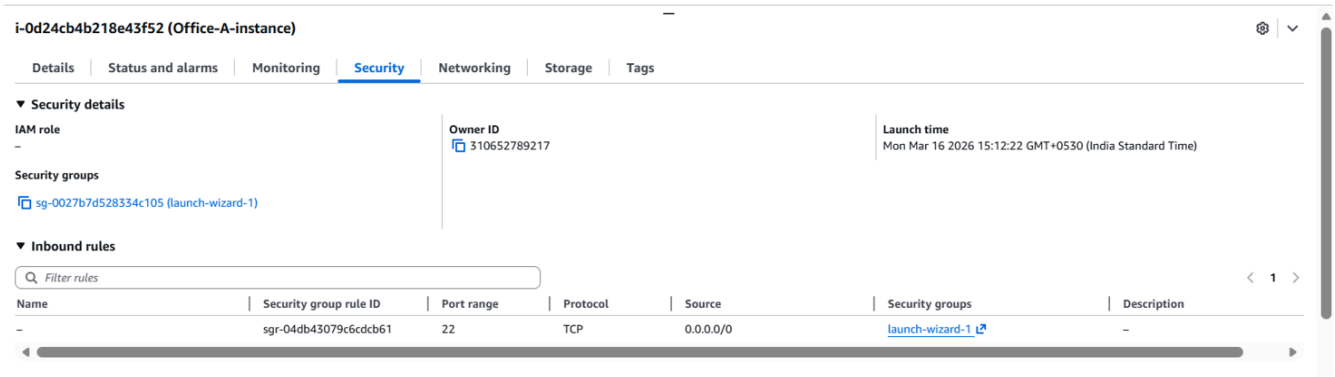
Security best practices implemented:

- Private subnets for internal resources
- Restricted security group rules
- SSH access control
- Least privilege access

HQ Environment Security Rules:-



Branch Office Security Rules:-



---

8. Cost Optimisation

Cost optimization strategies:

- Use of **t3.micro instances**
- Minimal infrastructure resources
- Avoid unnecessary AWS services

This keeps infrastructure cost minimal.

---

10. Implementation Timeline

Phase 1

- CIDR planning
- Subnet design

Phase 2

- VPC creation
- Subnet creation
- Route tables
- VPC peering

Phase 3

- EC2 deployment
- Connectivity testing
- Documentation

---

11. Continuous Improvement

11.1 Feedback and Retrospectives

Infrastructure feedback should focus on:

- Network scalability
- Security configuration
- Connectivity performance

11.2 Iterative Enhancements

Future improvements:

- Implement AWS Transit Gateway
- Infrastructure as Code using Terraform
- Monitoring dashboards

---

12. Conclusion

12.1 Summary

This project successfully implemented a **multi-region AWS network architecture using multiple VPCs and VPC peering connections**.

Key achievements:

- Multi-VPC design
- Secure subnet segmentation
- Cross-region connectivity
- Successful connectivity validation

12.2 Next Steps

Future enhancements may include:

- Infrastructure automation
- Container orchestration
- Advanced monitoring tools

---

13. Customer Acknowledgement

Name: _____

Title: _____

Signature: _____

Date: _____